

SIMATS ENGINEERING

ASSESSMENT TOOL 1

Name: V Sunil Kumar

Reg no: 192425292

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

Assessment Tool Weightage: 40%

Dr. Dumpala Prasanth

QUESTIONS: 10 Total Marks: 50

Annexure A

Descriptive Experiment Exam Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development

Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Code of Conduct:

I V Sunil Kumar/192425292_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V Sunil Kumar

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim

To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium, NumPy, Matplotlib, and verify the installation by executing a simple Reinforcement Learning program.

Algorithm

1. Install Anaconda Navigator.
2. Create a Python environment.
3. Install Gymnasium, TensorFlow, NumPy, and Matplotlib.
4. Import the required libraries.
5. Create a Gymnasium environment.
6. Display the observation and action spaces.

Python Code

```
import gymnasium as gym
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
env = gym.make("CartPole-v1")
print("Observation Space:", env.observation_space)
print("Action Space:", env.action_space)
env.close()
```

Result

The required Reinforcement Learning libraries were installed successfully and the CartPole environment was created successfully.

Experiment 2: Familiarization with OpenAI Gymnasium

Aim

To create and interact with Gymnasium environments.

Algorithm

1. Import Gymnasium.
2. Create FrozenLake environment.
3. Reset the environment.
4. Perform random actions.
5. Display state, reward, and termination status.

Python Code

```
import gymnasium as gym

env = gym.make("FrozenLake-v1", render_mode="human")

state, info = env.reset()

for i in range(10):

    action = env.action_space.sample()

    state, reward, terminated, truncated, info = env.step(action)

    print("State:", state, "Reward:", reward)

    if terminated or truncated:

        state, info = env.reset()

env.close()
```

Result

The agent interacted with the FrozenLake environment using random actions.

Experiment 3: Reinforcement Learning Framework

Aim

To implement the basic Reinforcement Learning framework.

Algorithm

1. Create environment.
2. Initialize state.
3. Select random action.
4. Perform action.
5. Receive reward.
6. Continue until episode ends.

Python Code

```
import gymnasium as gym

env = gym.make("CartPole-v1")

state, info = env.reset()

done = False

while not done:

    action = env.action_space.sample()

    state, reward, terminated, truncated, info = env.step(action)
```

```
print("Reward:", reward)

done = terminated or truncated

env.close()
```

Result

The RL agent interacted with the environment and received rewards until the episode ended.

Experiment 4: Markov Decision Process (MDP)

Aim

To simulate a simple Markov Decision Process.

Algorithm

1. Define states.
2. Define actions.
3. Define rewards.
4. Perform state transitions.
5. Display next state and reward.

Python Code

```
states = ["A", "B", "C"]

transitions = {
    "A": ("B", 5),
    "B": ("C", 10),
    "C": ("A", 2)
}

state = "A"

for i in range(5):
    next_state, reward = transitions[state]
    print(state, "->", next_state, "Reward:", reward)
    state = next_state
```

Result

The MDP simulation successfully demonstrated state transitions and rewards.

Experiment 5: Bellman Equation

Aim

To implement Bellman Value Update.

Algorithm

1. Initialize value.
2. Define reward.
3. Define discount factor.
4. Apply Bellman equation.
5. Print updated value.

Python Code

```
gamma = 0.9
reward = 10
next_value = 20
value = reward + gamma * next_value
print("Updated Value:", value)
```

Result

The Bellman equation successfully calculated the updated state value.

Experiment 6: Exploration vs Exploitation

Aim

To implement ϵ -Greedy strategy.

Algorithm

1. Initialize epsilon.
2. Generate random number.
3. Explore or exploit.
4. Print selected action.

Python Code

```
import random
epsilon = 0.2
best_action = 1
for i in range(10):
    if random.random() < epsilon:
        action = random.randint(0, 1)
        print("Explore:", action)
    else:
```

```
print("Exploit:", best_action)
```

Result

The ϵ -Greedy strategy successfully balanced exploration and exploitation.

Experiment 7: Multi-Armed Bandit Problem

Aim

To implement ϵ -Greedy Multi-Armed Bandit.

Algorithm

1. Initialize rewards.
2. Select action.
3. Update reward.
4. Repeat.
5. Display cumulative reward.

Python Code

```
import random
arms = [1, 2, 3]
total_reward = 0
for i in range(10):
    arm = random.choice(arms)
    reward = random.randint(0, 10)
    total_reward += reward
    print("Arm:", arm, "Reward:", reward)
print("Total Reward:", total_reward)
```

Result

The Multi-Armed Bandit algorithm successfully selected different actions and accumulated rewards.

Experiment 8: Reward Visualization

Aim

To visualize cumulative rewards using Matplotlib.

Algorithm

1. Create reward list.

2. Compute cumulative rewards.
3. Plot graph.
4. Display graph.

Python Code

```
import matplotlib.pyplot as plt
rewards = [1,2,3,2,4,5,3,6]
cumulative = []
total = 0
for r in rewards:
    total += r
    cumulative.append(total)
plt.plot(cumulative)
plt.title("Cumulative Reward")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.show()
```

Result

The cumulative rewards were successfully visualized.

Experiment 9: TensorFlow & Keras Neural Network

Aim

To build a simple neural network using TensorFlow and Keras.

Algorithm

1. Import TensorFlow.
2. Create Sequential model.
3. Add Dense layers.
4. Compile model.
5. Display summary.

Python Code

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
model = Sequential()
```



```
model.add(Dense(24, input_shape=(4,), activation='relu'))
model.add(Dense(24, activation='relu'))
model.add(Dense(2, activation='linear'))
model.compile(optimizer='adam', loss='mse')
model.summary()
```

Result

The neural network model was successfully created and compiled.

Experiment 10: Mini Reinforcement Learning Project (CartPole)

Aim

To create a basic Reinforcement Learning agent for the CartPole environment.

Algorithm

1. Create CartPole environment.
2. Reset environment.
3. Select random actions.
4. Receive rewards.
5. Display total reward.
6. End when episode finishes.

Python Code

```
import gymnasium as gym
env = gym.make("CartPole-v1")
state, info = env.reset()
total_reward = 0
done = False
while not done:
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    done = terminated or truncated
print("Total Reward:", total_reward)
env.close()
```

Result

The CartPole agent successfully interacted with the environment and accumulated rewards until the episode ended.